

Figure 1: The typical push notification system

```

android:label="@string/app_name" >
<intent-filter>
    <action android:name="android.intent.action.MAIN"

/>
    <category android:name="android.intent.category.
LAUNCHER" />
</intent-filter>
<activity>
<receiver
    android:name="com.google.android.gcm.
GCMBroadcastReceiver"
    android:permission="com.google.android.c2dm.
permission.SEND" >
    <intent-filter>
        <action android:name="com.google.android.c2dm.
intent.RECEIVE" />
        <action android:name="com.google.android.c2dm.
intent.REGISTRATION" />
        <category android:name="in.amolgupta.android.gcm"

/>
    </intent-filter>
</receiver>
<service android:name=".GCMIntentService" />
<activity
    android:name=".NoteList"
    android:label="@string/title_activity_note_list" >
</activity>
</application>
</manifest>

```

Write the service that extends *GCMBaseIntentService* and implements the functions *onRegistered()*, *onUnregistered()*, *onMessage()*, *onError()*, and *onRecoverableError()*. Next, you need to write the activity for this application. In the activity, you can call the functions from the jar file to check if the device is GCM enabled or not, and also to check if your manifest file has all the required configurations.

```

GCMRegistrar.checkDevice(this);
GCMRegistrar.checkManifest(this);

```

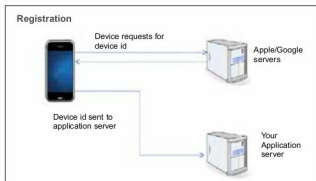


Figure 2: Registration

To register the device, call the function *GCMRegistrar.register(this, SENDER_ID)*; here, *SENDER_ID* is the 12-digit project ID from the API console.

Running this application would register it on the Google server, and return a device ID, which can be sent to the server using a Web service. For simplicity, here let us just copy it to the server-side code.

Last, you need to write the server-side code for the system. To send the push notifications, a Google Web service is called, which can be done with any programming language. Using the *gcm-server.jar* from the Android SDK, the implementation becomes very simple, and can be integrated into any Java-based system such as your Web services, desktop clients, server-side schedulers, etc. The following lines of code can be used once the JAR has been added to the class path:

```

Sender sender = new Sender("Server key");
ArrayList<String> devicesList = new ArrayList<String>();
//add the device ids from Android device
devicesList.add("<android device id>");
Message message = new Message.Builder().collapseKey("1")
    .timeToLive(3).delayWhileIdle(true)
    .addData("message", "message text!!")
    .build();
MulticastResult result = sender.send(message, devicesList, 1);

```

Executing this code would push a notification to the Android device. By implementing proper handling of the intents, you can show this message in the notification bar as an alert, or can initiate any other action in the application. There is a limit on the amount of data that can be sent via push notifications; therefore, it is recommended that instead of sending the complete message content, you should only notify the device that there is a new message available, and let it retrieve the actual message with another service call. 

By: Amol Gupta

The author works for Infosys Mobility, and has a keen interest in mobile and cloud computing. He likes to explore the various possibilities in the two fields, and how they complement each other. He can be reached at amolgupta1989@gmail.com and <http://twitter.com/amolgupta>.